



SAPIENZA
UNIVERSITÀ DI ROMA

INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E STATISTICA

Echidna Project

SECURITY IN SOFTWARE APPILCATION

Professor:

Friolo Daniele

Student:

Lorenzo Sabatino, XXXXXXXXXX

Contents

1	Introduction	2
1.1	Echidna	2
2	Part 1 — Marriage invariants	3
3	Part 2 — Tax allowance invariants	6
4	Part 3 — Age-aware invariants	7
5	Part 4 — Lottery invariants	9
6	Conclusion	11

1 Introduction

This project uses a modern property-based fuzzing approach to find logical and stateful bugs in Solidity smart contracts. Instead of writing a small number of example unit tests with specific inputs, you declare general properties (*invariants*) that the contract must always satisfy and then let an automated tool generate large numbers of sequences of transactions with many different inputs, searching for sequences that cause any property to fail. This is the core of property-based fuzzing.

Fuzzing is an automated random input generation: the fuzzer repeatedly calls functions with randomly chosen input values, trying to make the program crash or behave incorrectly.

This technique is particularly effective for smart contracts, which are stateful and interact with external callers and other contracts over time. Here, many vulnerabilities do not arise from a single function call, but from specific sequences of interactions that gradually move the contract into an invalid state. In this context, property-based fuzzing explores complex behaviors and detects bugs that traditional testing methods often miss.

1.1 Echidna

Echidna (<https://github.com/crytic/echidna>) is a property-based fuzzer for Ethereum smart contracts. It is designed with Solidity in mind and implements several features effective for contract testing:

- *ABI-driven test-generation.* Echidna inspects the contract ABI and uses that to generate sequences of transactions.
- *Property hooks.* Echidna expects the developer to provide one or more boolean functions whose names start with the prefix `echidna_`. This naming convention allows Echidna to distinguish invariants from regular contract logic. Each such function represents a property that should always hold and returns `true` when the property is satisfied. If any of these functions evaluates to `false` during execution, Echidna reports the corresponding input sequence as a counterexample.
- *Sequences of calls.* Echidna generates sequences of function calls (transactions) across the same contract instance, exploring how different sequences affect its state.
- *Integration with the Ethereum.* Echidna executes smart contract transactions: it can create addresses and call other contracts as part of its testing process.

Each `echidna_` function is intended to encode a single invariant that captures a property of the contract. To properly define and execute these properties I followed the following steps:

1. Implement the logic and add `echidna_` functions encoding the critical properties.
2. Run Echidna inside a Docker container with the following commands:

```
1 docker run -it -v $(pwd):/src ghcr.io/crytic/echidna/echidna
2 cd /src
3 echidna Taxpayer.sol --contract TestTaxpayer
4 echidna Lottery.sol --contract TestLottery
```

After that Echidna compiles the source and starts fuzzing.

3. If Echidna finds a property violation, it reports the failure and gives a counterexample (a short sequence of function calls and arguments that reproduce the failure).
4. Analyze the counterexample to understand the root cause, then modify the contract.
5. Finally, rerun Echidna to confirm that the invariant now holds and that no counterexample can be found after the fix.

2 Part 1 — Marriage invariants

The `Taxpayer.sol` contract stores basic personal information for on-chain “people,” including age, spouse, parents, and a tax allowance. I introduced the following invariants:

- `echidna_simmetric_marriage()` Marriage is a symmetric relationship. If one side stores the other but the other side does not, the system is in an inconsistent state.

```
1 function echidna_simmetric_marriage() public view returns (bool) {
2     if (spouse != address(0)) {
3         if (!isValidTaxpayer(spouse)) return true;
4         return Taxpayer(spouse).spouse() == address(this);
5     }
6     return true;
7 }
```

- `echidna_marriage_status()` The boolean variable `isMarried` must reflect whether a spouse pointer is set. Here the two representations of the same concept (a boolean flag and a pointer) must stay consistent. If they diverge, code that use only one of them can behave incorrectly.

```

1  function echidna_marriage_status() public view returns (bool) {
2      if (spouse != address(0)) {
3          return isMarried == true;
4      } else {
5          return isMarried == false;
6      }
7  }

```

- `echidna_no_self_marriage()` A contract must never record itself as its own spouse. Functions that iterate over spouse relationships, or assume two distinct parties, can break or behave incorrectly if the two parties are identical.
- `echidna_spouse_is_valid_taxpayer()` The spouse address must correspond to a deployed contract that implements the `Taxpayer` interface. This prevents later calls that treat the spouse as a `Taxpayer` from invoking unexpected behavior.
- `echidna_no_self_parent()` A contract must never record itself as its own parent.
- `echidna_parents_distinct()` If the contract stores two parent pointers, they must not be equal one to the other.

The screenshot in Figure 1 is the captured output from the failing Echidna run. It shows two failures:

- `echidna_marriage_status`: FAILED! with `ReturnFalse`. Call sequence in minimized counterexample: `TestTaxpayer.marry(0x0)`. Echidna found a sequence where spouse became `0x0` but `isMarried` was left set to true.
- `echidna_simmetric_marriage`: FAILED! with `ErrorRevert`. Call sequence in minimized counterexample: `TestTaxpayer.marry(0xdeadbeef)`. The contract attempted to set a non-contract address as spouse or otherwise caused an inconsistent state.

The `marry(address new_spouse)` implementation in the original file simply did:

```

1  spouse = new_spouse;
2  isMarried = true;

```

with no checks. This allows setting spouse to `address(0)` or to a non-contract address, which later functions attempt to call. Also here we are not ensuring the spouse's record is updated symmetrically. The function `divorce()` also simply set:

```

1  spouse = address(0);
2  isMarried = false;

```

with no attempt to update the spouse's state.

Because of these Echidna was able to construct tiny sequences that left the contract in an inconsistent state. To resolve the problems I applied these modifications:

```

root@c6dad01e8825: /src
lore@lore-debian: ~/uni/ssa/echidna_project  x  lore@lore-debian: ~/uni/ssa/echidna_project  x  root@c6dad01e8825: /src  x

[ Echidna 2.3.0 ]

Workers: 0/4      Unique instructions: 1485      Chain ID: -
Seed: 7822038686435400327      Unique codehashes: 1      Fetched contracts: 0/0
Calls/s: 10021      Corpus size: 8 seqs      Fetched slots: 0/0
Gas/s: 102287931      New coverage: 2s ago
Total calls: 50107/50000      Slither succeeded

Tests (4) [*]

echidna_marriage_status: FAILED! with ReturnFalse
Call sequence:
1. TestTaxpayer.marry(0x0)

echidna_simmetric_marriage: FAILED! with ErrorRevert
Call sequence:
1. TestTaxpayer.marry(0xdeadbeef)

Traces:
Log (15)
[2025-12-22 16:24:28.07] [Worker 0] Test limit reached. Stopping.
[2025-12-22 16:24:28.05] [Worker 2] Test limit reached. Stopping.
[2025-12-22 16:24:27.25] [Worker 1] Test limit reached. Stopping.
[2025-12-22 16:24:27.22] [Worker 3] Test limit reached. Stopping.
[2025-12-22 16:24:25.30] [Worker 1] New coverage: 1485 instr, 1 contracts, 8 seqs in corpus
[2025-12-22 16:24:24.73] [Worker 3] New coverage: 1485 instr, 1 contracts, 7 seqs in corpus
[2025-12-22 16:24:23.71] [Worker 3] New coverage: 1485 instr, 1 contracts, 6 seqs in corpus

Campaign complete, C-c or esc to exit

```

Figure 1: Echidna marriage invariants before the fix.

- Introduce `isValidTaxpayer(address addr)` helper that encapsulates every low-level check you want to run before treating an address as a `Taxpayer` contract. It rejects the zero address immediately, it confirms there is deployed code at the address and it exercises a lightweight, read-only `Taxpayer` probe.
- Redesign `marry(new_spouse)` so that it enforces several checks and then attempts to establish the relationship on both sides:
 - The proposed spouse must not be the zero address.
 - The caller must not already be married.
 - The proposed spouse must not be the contract itself.
 - The proposed spouse must pass `isValidTaxpayer` so you only accept real `Taxpayer` contracts.
 - The proposed spouse must either be unmarried or already point at the caller.
- Redesign `divorce()` to be explicit about the precondition and symmetry when divorcing. To safely divorce a marriage first, the contract updates its own state immediately by clearing the spouse pointer and setting `isMarried` to `false`. Performing changes before any external calls prevents attacks from a malicious spouse contract that might attempt to re-enter while the contract still appears married. Once the local state has been safely cleared, the function attempts to contact the spouse: the function invokes the corresponding `divorce()` to ensure that the relationship is symmetrically removed. If this external call fails the local state remains consistent.

- Rather than having one contract change another contract's internal allowance variable, I introduced a `receiveAllowance(uint change)` on every `Taxpayer` that serves as the entry point for receiving allowance transfers.

```

1 function receiveAllowance(uint change) public {
2     // Only my spouse can send me allowance.
3     require(msg.sender == spouse, "Only spouse can transfer allowance");
4     require(spouse != address(0), "Must have valid spouse");
5     tax_allowance = tax_allowance + change;
6 }

```

So now `transferAllowance(change)` calls the spouse's `receiveAllowance(change)` instead of directly change its internal fields; `receiveAllowance(change)` only accepts allowance increases or decreases from the spouse, and it performs any necessary validation internally. This eliminates situations where one contract force-writes into another's state and keeps each contract responsible for its own invariants.

The results in Figure 2 show the marriage-related invariants now pass.

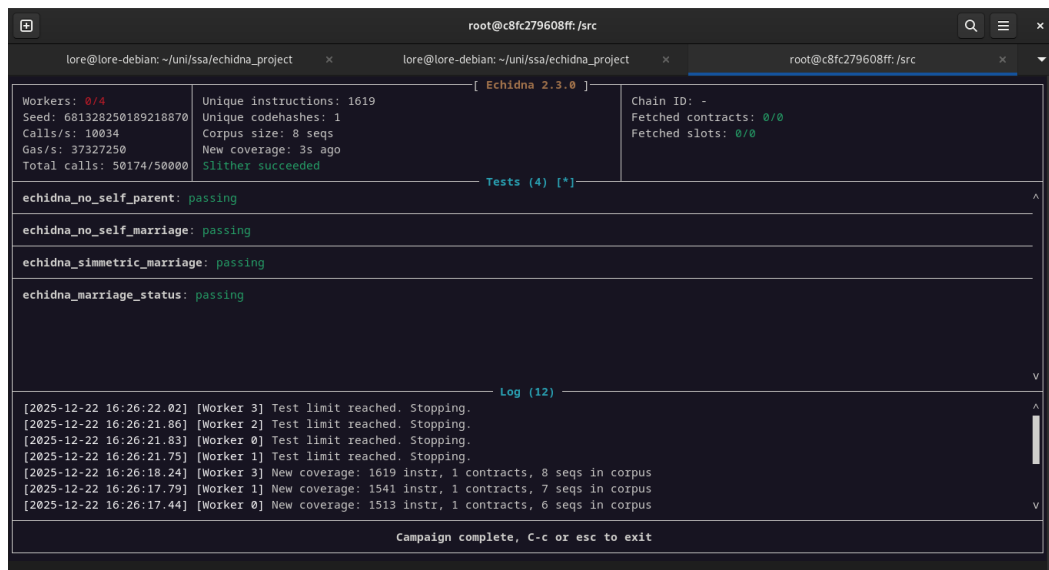


Figure 2: Echidna marriage invariants after the fix.

3 Part 2 — Tax allowance invariants

The tax system in the `Taxpayer` contract implements a per-person income tax allowance (deduction):

- Every person has a default tax allowance of 5000 (`DEFAULT_ALLOWANCE`).
- Married persons can shift allowances between them, as long as the sum of their allowances remains the same.

The invariant `echidna_tax_allowance()` ensures that single taxpayers always retain the minimum base allowance. For married couples, it enforces conservation of the total allowance, assuming the initial baseline is $2\times$ defaults.

```

1 function echidna_tax_allowance() public view returns (bool) {
2     if (spouse != address(0)) {
3         uint256 myAllowance = tax_allowance;
4         uint256 spouseAllowance = Taxpayer(spouse).getTaxAllowance();
5
6         return (myAllowance + spouseAllowance) == (2 * DEFAULT_ALLOWANCE);
7     } else {
8         return tax_allowance >= DEFAULT_ALLOWANCE;
9     }
10 }

```

As already said, allowance receipt was isolated by introducing `receiveAllowance(uint change)`, which can only be called by the contract's spouse.

After applying the changes described above, Echidna reports that all allowance invariants hold. No counterexamples were found during fuzzing, as illustrated in Figure 3.

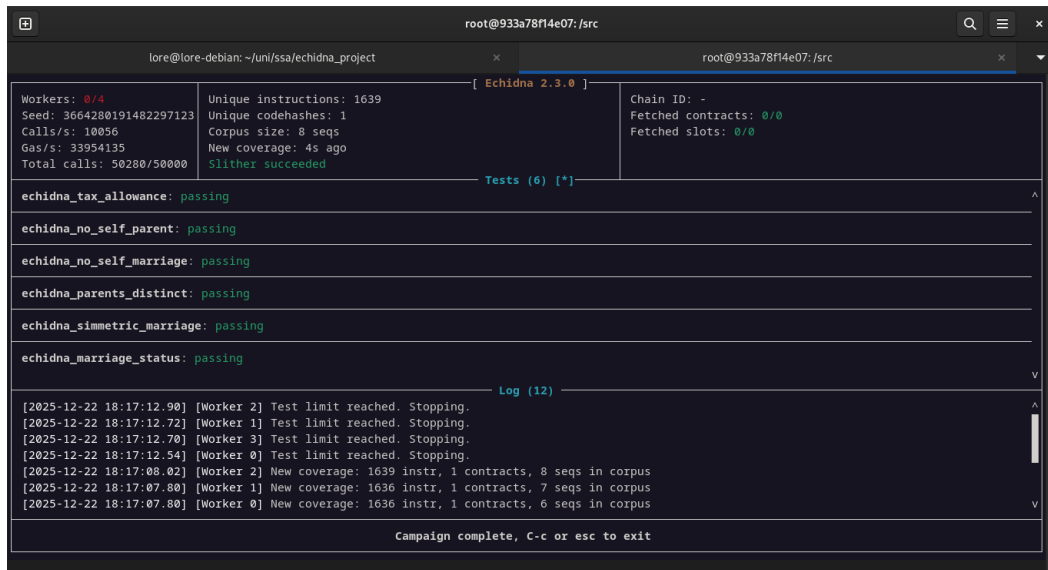


Figure 3: Echidna tax allowance invariants after the fix.

4 Part 3 — Age-aware invariants

The government introduced a rule: people aged 65 or older must have a higher tax allowance of 7000 (`ALLOWANCE_OAP`) instead of the default 5000 (`DEFAULT_ALLOWANCE`). Earlier rules still apply: spouses may transfer allowance among themselves but the total allowance must remain correct. For this part I added:

- **receivedLotteryBonus** Here we need to distinguish a taxpayer who received an absolute allowance of 7000 from the Lottery from one who automatically reached 65 and therefore became eligible for 7000. This flag helps the invariants reason about the history of the account (e.g., to avoid double counting when someone receives an award then turns 65).
- In `haveBirthday()`, when an individual reaches age 65 and has not previously received a lottery bonus, the contract automatically increases `tax_allowance` by `ALLOWANCE_OAP - DEFAULT_ALLOWANCE`. This ensures that taxpayers automatically receive the senior allowance on his 65th birthday.

```

1 function haveBirthday() public {
2     age++;
3     if (age == 65 && !receivedLotteryBonus) {
4         tax_allowance = tax_allowance + (ALLOWANCE_OAP - DEFAULT_ALLOWANCE);
5     }
6 }

```

- Function `setTaxAllowance()` is now used exclusively to allow the Lottery to assign an absolute allowance. Previously many contracts could call `setTaxAllowance` and produce writes that could break invariants.
- `echidna_total_allowance_preserved_age()` When age-based rules are considered, the minimum allowance is computed based on whether each individual is 65 or older. For married couples, the sum of the actual allowances must equal the sum of the age-based minimums. For single taxpayers, the allowance must be at least their age-based minimum.

This invariant refines the default-based rule by adding older-age allowances. It prevents the system from reducing total allowances below the legally required age-based minimums.

```

1 function echidna_total_allowance_preserved_age() public view returns (bool) {
2     uint myMin = age >= 65 ? ALLOWANCE_OAP : DEFAULT_ALLOWANCE;
3
4     if (isMarried && spouse != address(0) && isValidTaxpayer(spouse)) {
5         Taxpayer sp = Taxpayer(spouse);
6         uint spMin = sp.getAge() >= 65 ? ALLOWANCE_OAP : DEFAULT_ALLOWANCE;
7
8         uint totalMin = myMin + spMin;
9         uint total = tax_allowance + sp.getTaxAllowance();
10        return total == totalMin;
11    } else {
12        return tax_allowance >= myMin;
13    }
14 }

```

- `echidna_allowance_never_zero()` The tax allowance must always satisfy `tax_allowance > 0`. This guards against bugs that might zero out allowances, which would violate fundamental assumptions of the tax system.
- `echidna_lottery_bonus_consistency()` and `echidna_senior_minimum_allowance()` These invariants impose additional constraints related to lottery-awarded allowances and senior minimum allowances. In particular, they account for interactions with the `Lottery` contract, which can assign a fixed allowance of 7000 via `setTaxAllowance`.

With these special cases, the invariants ensure that external interactions (such as lottery rewards) do not violate global allowance consistency or age-based minimum requirements.

- `echidna_senior_minimum_allowance()`: Ensures single seniors have at least the senior minimum.

After the changes I ran Echidna with the new age-aware invariants and receive back the results showed in Figure 4.

```

root@ec64d3532c01: /src
lore@lore-debian: ~/ssa/echidna_project x root@ec64d3532c01: /src

[ Echidna 2.3.0 ]
Workers: 0/4 Unique instructions: 1739 Chain ID: -
Seed: 7658571099622309342 Unique codehashes: 1 Fetched contracts: 0/0
Calls/s: 8382 Corpus size: 8 seqs Fetched slots: 0/0
Gas/s: 25104797 New coverage: 4s ago
Total calls: 50292/50000 Slither succeeded

Tests (7) [*]
echidna_tax_allowance: passing
echidna_no_self_parent: passing
echidna_no_self_marriage: passing
echidna_parents_distinct: passing
echidna_symmetric_marriage: passing
echidna_total_allowance_preserved_age: passing

Log (12)
[2025-12-22 18:35:01.65] [Worker 2] Test limit reached. Stopping.
[2025-12-22 18:35:01.33] [Worker 3] Test limit reached. Stopping.
[2025-12-22 18:35:01.27] [Worker 1] Test limit reached. Stopping.
[2025-12-22 18:35:01.19] [Worker 0] Test limit reached. Stopping.
[2025-12-22 18:34:57.18] [Worker 0] New coverage: 1739 instr, 1 contracts, 8 seqs in corpus
[2025-12-22 18:34:56.74] [Worker 0] New coverage: 1711 instr, 1 contracts, 7 seqs in corpus
[2025-12-22 18:34:56.34] [Worker 2] New coverage: 1633 instr, 1 contracts, 6 seqs in corpus

Campaign complete, C-c or esc to exit

```

Figure 4: Echidna age-aware invariants after the fix.

5 Part 4 — Lottery invariants

The Lottery contract implements a periodic lottery where participating taxpayers can win an increased allowance (7000) for a round. A common lottery is:

1. `startLottery()` sets `startTime`, `revealTime` and `endTime`.

2. `commit(bytes32 y)` participants register $y = \text{keccak256}(x)$ where x is their secret value.
3. `reveal(uint256 rev)` participants reveal x and the contract checks `keccak256(x)` corresponds to `commits[msg.sender]` and records the revealed x .
4. `endLottery()` after the end phase, computes the winner as $j = (x_1 + \dots + x_n) \% n$ and sets that winner's allowance to 7000.

Knowing that, I added the following:

- `isValidTaxpayer(address addr)` Implemented a local check inside `Lottery.sol` to make sure addresses recorded as participants are deployed `Taxpayer` contracts. When the lottery picks a winner it invokes `Taxpayer(...).setTaxAllowance(7000)`. So if the chosen entry is not a valid `Taxpayer`, the call may revert or cause undefined behavior.
- `proxy_commit(uint256 val)` A helper that computes `bytes32` hash. Echidna picks raw numeric values as fuzz inputs; we provide a helper that computes the commitment locally and calls `commit` with the correct hash, following the guidance in the assignment *“craft input to the commit function by first computing keccak256 on the echidna testing input”*.

```

1 function proxy_commit(uint256 val) public {
2     bytes32 hash = keccak256(abi.encode(val));
3     this.commit(hash);
4 }

```

- `hasRevealed` is used to track participation/reveals and to prevent duplicates or double-reveals. Contracts must ensure a participant cannot appear more than once in the revealed array, because duplicates could bias the winner selection.
- `echidna_no_duplicates_in_reveal()` The revealed array must not contain duplicate addresses. (To avoid gas explosion we limit checks to the first 10 entries.)

```

1 function echidna_no_duplicates_in_reveal() public view returns (bool) {
2     uint limit = revealed.length;
3     if (limit > 10) limit = 10;
4
5     for (uint i = 0; i < limit; i++) {
6         for (uint j = i + 1; j < limit; j++) {
7             if (revealed[i] == revealed[j]) {
8                 return false;
9             }
10        }

```

```

11     }
12     return true;
13 }

```

- `echidna_state_reset_correctly()` `endLottery()` computes the total (sum of revealed values), selects winner and set its allowance. Then it resets `startTime`, `revealTime` and `endTime` to zero. To avoid leaving stale state, this invariant check that when `startTime` is zero (meaning no active lottery), the revealed array is empty and `hasRevealed` flags are not left set.

```

1 function echidna_state_reset_correctly() public view returns (bool) {
2     if (startTime == 0) {
3         if (revealed.length != 0) return false;
4
5         uint checkLimit = revealed.length > 10 ? 10 : revealed.length;
6         for (uint i = 0; i < checkLimit; i++) {
7             if (hasRevealed[revealed[i]]) return false;
8         }
9     }
10    return true;
11 }

```

- `echidna_revealed_are_taxpayers()` Every address in revealed must pass `isValidTaxpayer`. This prevents crashes when `endLottery()` calls `setTaxAllowance` on the winner.
- `echidna_reveal_requires_commit()` Every participant recorded in revealed must have a non-zero commit and must be marked as revealed in `hasRevealed`.
- `echidna_valid_timeline()` Ensures that if `startTime > 0` then `revealTime > startTime` and `endTime > revealTime`.
- `echidna_revealed_flag_consistent()` Checks that every address in revealed must have `hasRevealed` true.

After adding the invariants and the helper functions, Echidna's showed the invariants passing (see Figure 5)

6 Conclusion

This project demonstrates how property-based fuzzing with Echidna can be used to reason about the correctness of smart contracts that encode real-world policies. We expressed the intended behavior of the system as invariants and let Echidna automatically search for sequences of transactions that could violate them.

```
root@8db8cec8c736:/src
[ Echidna 2.3.0 ]
Workers: 0/4 Unique instructions: 1051 Chain ID: -
Seed: 3191370261573989949 Unique codehashes: 1 Fetched contracts: 0/0
Calls/s: 7179 Corpus size: 7 seqs Fetched slots: 0/0
Gas/s: 92600225 New coverage: 6s ago
Total calls: 50253/50000 Slither succeeded

Tests (6) [**]
echidna_state_reset_correctly: passing
echidna_reveal_requires_commit: passing
echidna_valid_timeline: passing
echidna_revealed_are_taxpayers: passing
echidna_revealed_flag_consistent: passing
echidna_no_duplicates_in_reveal: passing

Log (11)
[2025-12-23 17:56:12.47] [Worker 2] Test limit reached. Stopping.
[2025-12-23 17:56:12.36] [Worker 3] Test limit reached. Stopping.
[2025-12-23 17:56:12.34] [Worker 0] Test limit reached. Stopping.
[2025-12-23 17:56:12.18] [Worker 1] Test limit reached. Stopping.
[2025-12-23 17:56:05.79] [Worker 0] New coverage: 1051 instr, 1 contracts, 7 seqs in corpus
[2025-12-23 17:56:05.69] [Worker 3] New coverage: 1048 instr, 1 contracts, 6 seqs in corpus
[2025-12-23 17:56:05.69] [Worker 0] New coverage: 1048 instr, 1 contracts, 5 seqs in corpus

Campaign complete, C-c or esc to exit
```

Figure 5: Echidna lottery invariants after the fix.

This project shows the value of property-based fuzzing for smart contracts: invariants force developers to define the system behavior, fuzzing discovers the bugs, and the Echidna counterexamples guide the code fixes. So, in this context, Echidna provides a powerful tool for improving the reliability and security of Ethereum smart contracts before deployment.